PROCEDIMENTO DE CONFIGURAÇÃO DEVSECOPS: INTEGRAÇÃO GITHUB COM GITHUB ADVANCED SECURITY (GAS)

INTRODUÇÃO

Este guia detalha o procedimento para configurar um ambiente de análise estática de código, como parte da disciplina de Cultura e Práticas DevSecOps, integrando a ferramenta GitHub ao GitHub Advanced Security. Ao final deste procedimento, você terá criado as contas necessárias, configurado um repositório de exemplo e executado sua primeira análise de vulnerabilidades.

IMPORTANTE: Se você já realizou o Laboratório 1 e Laboratório 2, a **seção 1 não é necessário realizar.**

IMPORTANTE: O procedimento a seguir considera a interface fornecida pelos fabricantes na data corrente, portanto, imagens referentes às telas, não serão fornecidas devido a mudanças constante das interfaces Web. Em caso de mudanças no processo, por favor, verifique o processo semelhante.

IMPORTANTE: Você deve possuir o Git Bash no Microsoft Windows ou o Git instalado no Linux para conduzir este procedimento. Favor consultar o **Apêndice do Laboratório 01**.

IMPORTANTE: O repositório que será criado deverá ser público.

PROCEDIMENTO TÉCNICO

1. Criação da Conta no GitHub

O GitHub é o SCM (Source Code Management) que será utilizado para este procedimento, a plataforma onde o código-fonte dos projetos de software são armazenados, versionados e gerenciados de forma colaborativa. Para tanto:

1. **Acesse o site do GitHub**
    - Abra seu navegador Web e acesse: https://github.com/join
2. **Preencha os Dados de Cadastro**
    - Você pode realizar SSO (Single Sign-On) com sua conta do Google, por exemplo. Ou,
    - Siga as instruções na tela, informando um nome de usuário, um endereço de e-mail válido e criando uma senha segura.
3. **Verifique seu E-mail**
    - O GitHub enviará um e-mail de verificação para o endereço cadastrado. Acesse sua caixa de entrada e clique no link de confirmação para ativar sua conta.
4. **Finalize a Configuração**

- Após a verificação, você pode personalizar seu perfil. Para os fins deste guia, a configuração padrão é suficiente.

2. Entendendo o GitHub Advanced Security (GAS)

Antes de configurar, requer-se compreender uma visão geral do GitHub Advanced Security (GAS). O GAS não é uma ferramenta única, mas um conjunto de recursos de segurança integrados ao GitHub. Para este Laboratório, focaremos nos três pilares principais:

1. **Code Scanning (Análise de Código)**
    - É a ferramenta de Análise Estática de Segurança de Aplicação (SAST). Ele usa um motor chamado **CodeQL** para "ler" seu código-fonte e encontrar padrões de vulnerabilidades (como SQL Injection, Cross-Site Scripting (XSS), etc.) antes mesmo que o código seja executado.
2. **Secret Scanning (Análise de Segredos)**
    - É um módulo que monitora e procura por segredos expostos (chaves de API, senhas, tokens) que tenham sido acidentalmente (ou não) imputados como parte do processo de commit no código. Ele executa automaticamente em cada *push* e alerta se encontrar algo que pareça uma credencial.
3. **Dependabot (Análise de Dependências)**
    - É a ferramenta de Análise de Composição de Software (SCA). Ele escaneia as bibliotecas e pacotes de terceiros que seu projeto utiliza (entre eles, o pom.xml, package.json, requirements.txt) e alerta se alguma delas tiver uma vulnerabilidade conhecida (CVE). Ele pode, inclusive, sugerir o *pull request* de correção, que requer avaliar previamente o processo.

Importante: Em repositórios públicos (como o caso deste Laboratório), a maioria dessas funcionalidades (Code Scanning, Secret Scanning, Dependabot) é gratuita e está disponível. Em repositórios privados, elas fazem parte da licença paga do GitHub Advanced Security (GHAS) no plano Enterprise.

3. Configurando o Repositório de Exemplo (OWASP WebGoat)

Este Laboratório adota um projeto que é intencionalmente vulnerável. Há diversos projetos, mas adotaremos o **OWASP WebGoat**.

Em vez de criar um repositório do zero, será realizado um "Fork" (uma cópia) do projeto oficial para a sua conta.

1. Acesse o repositório oficial do WebGoat: https://github.com/WebGoat/WebGoat
2. No canto superior direito da página, clique no botão **"Fork"**.
3. O GitHub perguntará onde você quer criar o fork. Selecione a sua conta pessoal.
4. Aguarde alguns segundos. O GitHub irá copiar o projeto inteiro para a sua conta. Você será redirecionado para https://github.com/<SEU-USUARIO>/WebGoat.

4. Habilitando o GitHub Advanced Security (GAS) no seu Repositório

Agora requer-se ativar as ferramentas de segurança no repositório que você acabou de realizar o "fork".

1. Vá para a página do seu repositório WebGoat.
2. Clique na aba **"Settings"** (Configurações), localizada acima do nome do repositório (geralmente, como parte dos menus).
3. Nos menus do painel à esquerda, em Secuirity, selecione **"Advanced Security "**.

Aqui, serão as opções típicas para ativar o conjunto de ferramentas.

4.1 Habilitando Secret Scanning e Dependabot

Estas são as mais comuns na seção **Advanced Security**:

- **Private vulnerability reporting:** encontre esta seção e clique em **"Enable"** (Habilitar). Após habilitar, o botão ficará como "**Disable**".
- **Dependency graph:** encontre esta seção e clique em **"Enable"** (Habilitar).
- **Automatic dependency submission:** encontre esta seção e clique em **"Enable"** (Habilitar).

Estas são as mais comuns na seção **Dependabot**:

- **Dependabot alerts:** encontre esta seção e clique em **"Enable"** (Habilitar).
- **Dependabot security updates:** encontre e clique em **"Enable"**. (Isso permite que o Dependabot crie Pull Requests para corrigir as vulnerabilidades que encontrar).

4.2 Configurando o Code Scanning (com CodeQL)

Esta é a etapa principal. O Code Scanning é executado como uma **GitHub Action** (um fluxo de automação). Será necessário criar o arquivo que diz ao GitHub como executar o scan.

1. No seu repositório WebGoat, clique na aba principal **"Security"** (Segurança).
2. No menu à esquerda da aba "Security", clique em **"Advanced Security"**.
3. Você verá uma seção Code Scanning/Tools. Procure por "CodeQL Analysis" e clique no botão **"Set up"** ao lado dele, selecione **Default**, e em seguida, selecione "**Enable CodeQL**". Isto fará com que automaticamente, o scan seja realizado. Os resultados você poderá acompanhar no menu **Security/Vulnerability Alerts**, nas guias **Dependabot** e **Code scanning**.
4. Se você adotar o modo **Advanced**, será necessário desabilitar o modo Default. Em **Set up**, selecione Advanced, e então, o GitHub abrirá um editor de texto com um arquivo de yaml em WebGoat/.github/workflows/codeql.yml. Você poderá editar este arquivo se for necessário.

5. Executando os Scans e Analisando os Resultados

5.1 Acionando o Primeiro Scan

Quando você habilitou o CodeQL, houve o "commit" do arquivo codeql.yml (seção 4.2), você fez um *push* para o branch master. Isso **automaticamente acionou** o workflow do CodeQL.

1. No seu repositório do WebGoat, clique na aba **"Actions"**.
2. Você verá um workflow chamado **"CodeQL Setup"** em andamento (cor amarela).
3. Clique nele para acompanhar o progresso. O WebGoat é um projeto grande, então o primeiro scan pode demorar de 5 a 10 minutos. Aguarde ele ficar verde (concluído com sucesso).

5.2 Revisando o Painel de Segurança

Assim que o workflow "CodeQL" terminar, observe os achados no resultado.

1. Volte para a aba **"Security"** do seu repositório.
2. Observe que o painel, que antes estava vazio, agora possivelmente, deve possuir alertas.

Explore as sub-abas à esquerda:

- **Code scanning:** aqui estarão todas as vulnerabilidades de SAST encontradas pelo CodeQL. O WebGoat poderá conter dezenas delas (lembre-se, ele é intencionalmente vulnerável).
    - Clique em um alerta. O GitHub mostrará a linha exata do código, explicará a vulnerabilidade e, o mais importante, mostrará o fluxo de dados ("Show paths") de onde a vulnerabilidade começou (a *source*) até onde ela causou o impacto (o *sink*). Na tela inicial, você encontrará o link Show Path, e então, no canto superior direito, apresentará o Source e no final, o Sink. Em desenvolvimento seguro, **Source** e **Sink** são termos importantes, **por favor, pesquise sobre estes termos, muito comuns em revisão de segurança de código**.
- **Dependabot:** esta guia mostrará todas as bibliotecas de terceiros (dependências) que o WebGoat usa e que possuem vulnerabilidades conhecidas. Você encontrará a versão afetada (**Affectecd versions**) e a versão com a correção (**Patched version**).
- **Secret scanning:** esta guia (possivelmente) estará vazia, pois o projeto do WebGoat não iria realizar o "commit" de segredos reais.

5.3 (Opcional) Testando o Secret Scanning

Pode-se forçar um alerta de segredo para observar o Secret Scanning realizando a análise:

1. Vá para a aba **"Code"** do seu repositório.
2. Clique em **"Add file"** > **"Create new file"**.
3. Dê o nome ao arquivo de teste-secret-scanning.txt.
4. No conteúdo do arquivo, cole uma string que parece uma chave de API. O GitHub tem padrões para isso. Use este exemplo de formato de chave da AWS:

```
AKIAIOSFODNN7EXEMPLO
```

5. Faça o "Commit" deste novo arquivo.
6. Aguarde 1 ou 2 minutos e volte para a aba **"Security"** > **"Secret scanning"**.
7. Você poderá observar um alerta informando que uma "AWS Access Key ID" foi encontrada!

Por favor, não se esqueça de deletar este arquivo depois do teste!

CONCLUSÃO

Você configurou o GitHub Advanced Security, executou uma análise de vulnerabilidades com CodeQL e aprendeu a visualizar os resultados.